

# SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

## COURSE OUTCOME COVERED

**CO2:** Design Finite Automata DFA, NFA,  $\epsilon$ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 Total Marks:25 Duration : 1 Hour**

## Design Task

### **Code of Conduct:**

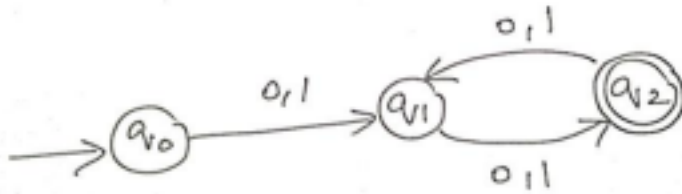
*I \_Pusalapati Chandu(192372119)\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:** 

## SIMATS ENGINEERING ASSESSMENT TOOL

1. Design the regular expression for the language represented by the given



automata

2. Consider the following grammar

$S \rightarrow aSc|T|U|e$

$T \rightarrow bTc|e$

$U \rightarrow aUb|e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

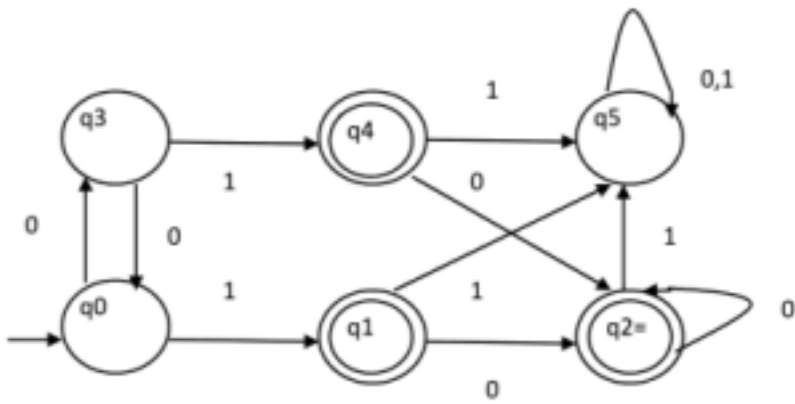
3. Design a non deterministic finite automata with  $\epsilon$ -transitions to represent the language denoted by the regular expression

$00^*1+11^*0$

4. I) Show that if  $L_1$  and  $L_2$  are regular then  $(L_1+L_2)$  also regular

li) Is the language  $\{a^p | p \text{ is prime}\}$  regular? Justify

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



# Question 1 — Regular Expression for the Given Automaton

Design the regular expression for the language represented by the given automaton (q0 = start; q1 = intermediate; q2 = accepting state, double circle).

## Step 1: The Given Automaton

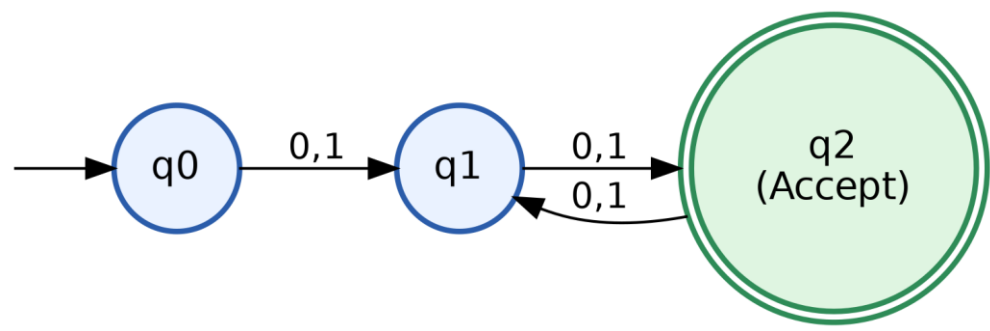


Figure 1.1 — Given automaton:  $q0 \rightarrow (0,1) \rightarrow q1 \rightarrow (0,1) \rightarrow q2$ , and  $q2 \rightarrow (0,1) \rightarrow q1$

## Step 2: Read Off the Transitions

| State      | On 0 | On 1 | Type           |
|------------|------|------|----------------|
| q0 (Start) | q1   | q1   | Non-Final      |
| q1         | q2   | q2   | Non-Final      |
| q2         | q1   | q1   | Final (Accept) |

### ***Step 3: Set Up Arden's Equations***

For every state, write an equation describing all strings that reach it from the start, using Arden's Theorem: if  $R = Q + RP$ , then  $R = QP^*$  (provided  $R$  does not appear in  $Q$ ).

- $q_0$  has no incoming edges other than the start arrow, so:  $q_0 = \epsilon$
- $q_1$  is reached from  $q_0$  on  $(0+1)$  and from  $q_2$  on  $(0+1)$ :  $q_1 = q_0(0+1) + q_2(0+1)$
- $q_2$  is reached from  $q_1$  on  $(0+1)$  only:  $q_2 = q_1(0+1)$

### ***Step 4: Solve the Equations***

Substitute  $q_0 = \epsilon$  into the equation for  $q_1$ :

$$q_1 = (0+1) + q_2(0+1)$$

Substitute  $q_2 = q_1(0+1)$  into this equation:

$$q_1 = (0+1) + q_1(0+1)(0+1) = (0+1) + q_1(0+1)^2$$

This is exactly the form  $R = Q + RP$  with  $Q = (0+1)$  and  $P = (0+1)^2$ . Applying Arden's Theorem ( $R = QP^*$ ):

$$q_1 = (0+1) \cdot ((0+1)(0+1))^*$$

Now substitute back to get  $q_2$  (the only accepting state, so  $q_2$  IS the required regular expression):

$$q_2 = q_1(0+1) = (0+1) \cdot ((0+1)(0+1))^* \cdot (0+1)$$

### ***Step 5: Final Regular Expression***

$$R = (0+1) ((0+1)(0+1))^* (0+1)$$

This can equivalently be written using the '+' (one-or-more) operator on the 2-symbol block as:

$$R = ((0+1)(0+1))^+$$

### ***Step 6: Verification / Justification***

- Meaning:  $R$  accepts exactly those binary strings whose length is EVEN and at least 2 — i.e., the string must be built entirely out of 2-symbol blocks, with at least one block present.
- Trace check — string "01" (length 2):  $q_0 \rightarrow(0) \rightarrow q_1 \rightarrow(1) \rightarrow q_2$  (Accept) ✓ matches  $R$  with one block.
- Trace check — string "110" (length 3, odd):  $q_0 \rightarrow(1) \rightarrow q_1 \rightarrow(1) \rightarrow q_2 \rightarrow(0) \rightarrow q_1$  — ends in  $q_1$ , NOT accepted, and  $R$  correctly excludes odd-length strings since every block in  $R$  contributes exactly 2 symbols.
- Trace check — empty string  $\epsilon$  (length 0): machine stays at  $q_0$ , which is non-final — correctly rejected, and  $R$  also cannot generate  $\epsilon$  because '+' (one-or-more) requires at least one block.

Justification: The state-elimination method using Arden's Theorem systematically converts the automaton's transition structure into a closed-form regular expression by solving a system of linear language-equations —

this is the standard, provably correct technique for the "automaton  $\rightarrow$  regular expression" direction of Kleene's equivalence theorem.

## Question 2 — Grammar Analysis

---

Grammar:  $S \rightarrow aSc \mid T \mid U \mid \varepsilon$ ,  $T \rightarrow bTc \mid \varepsilon$ ,  $U \rightarrow aUb \mid \varepsilon$

### *Part A: What Language Does the Grammar Define?*

Since  $S \rightarrow aSc$  can be applied  $n \geq 0$  times before bottoming out, every derivation from  $S$  has the shape:

$$S \Rightarrow a^n (\text{T or U or } \varepsilon) c^n$$

Now expand each bottoming-out case:

- Case  $S \rightarrow \varepsilon$  directly: gives  $a^n c^n$  (i.e.,  $i = n$  a's,  $j = 0$  b's,  $k = n$  c's, so  $i = k, j = 0$ )
- Case  $S \rightarrow T$ , and  $T \rightarrow bTc \mid \varepsilon$  generates  $b^m c^m$ : gives  $a^n b^m c^{m+n}$  → here  $i = n, j = m, k = m+n$ , so the condition is  $k = i + j$  (any  $i, j \geq 0$ )
- Case  $S \rightarrow U$ , and  $U \rightarrow aUb \mid \varepsilon$  generates  $a^p b^p$ : gives  $a^{n+p} b^p c^n$  → here  $i = n+p, j = p, k = n$ , so the condition is  $i = j + k$  (any  $j, k \geq 0$ )

The direct  $\varepsilon$ -case ( $i = k, j = 0$ ) is a special case of both the T-branch (set  $m = 0$ ) and the U-branch (set  $p = 0$ ), so it does not add anything new. Therefore, the language generated is:

$$L(S) = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (k = i + j \text{ OR } i = j + k) \}$$

In words: the string must consist of a block of a's, then a block of b's, then a block of c's (in that fixed order), where EITHER the number of c's equals the combined number of a's and b's, OR the number of a's equals the combined number of b's and c's.

### *Part B: Derivation Tree for "aaabbc"*

Count symbols in "aaabbc":  $i = 3$  a's,  $j = 2$  b's,  $k = 1$  c. Check: is  $i = j + k$ ? →  $3 = 2 + 1$  ✓ True. So this word is generated through the U-branch, with  $n = k = 1$  and  $p = j = 2$ .

Step-by-step derivation:

- $S \Rightarrow aSc$  (apply  $S \rightarrow aSc$  once,  $n = 1$ )
- $S \Rightarrow aUc$  (bottom out via  $S \rightarrow U$ )
- $U \Rightarrow aUb \Rightarrow aaUbc$  (apply  $U \rightarrow aUb$ , giving  $aaUbc \rightarrow$  written as  $a(aUb)c = aaUbbc$  — see tree)
- $U \Rightarrow aUb$  again (apply  $U \rightarrow aUb$  a second time,  $p = 2$  total)
- $U \Rightarrow \varepsilon$  (bottom out via  $U \rightarrow \varepsilon$ )
- Result:  $a(a(a\varepsilon b)b)c = aaabbc$  ✓

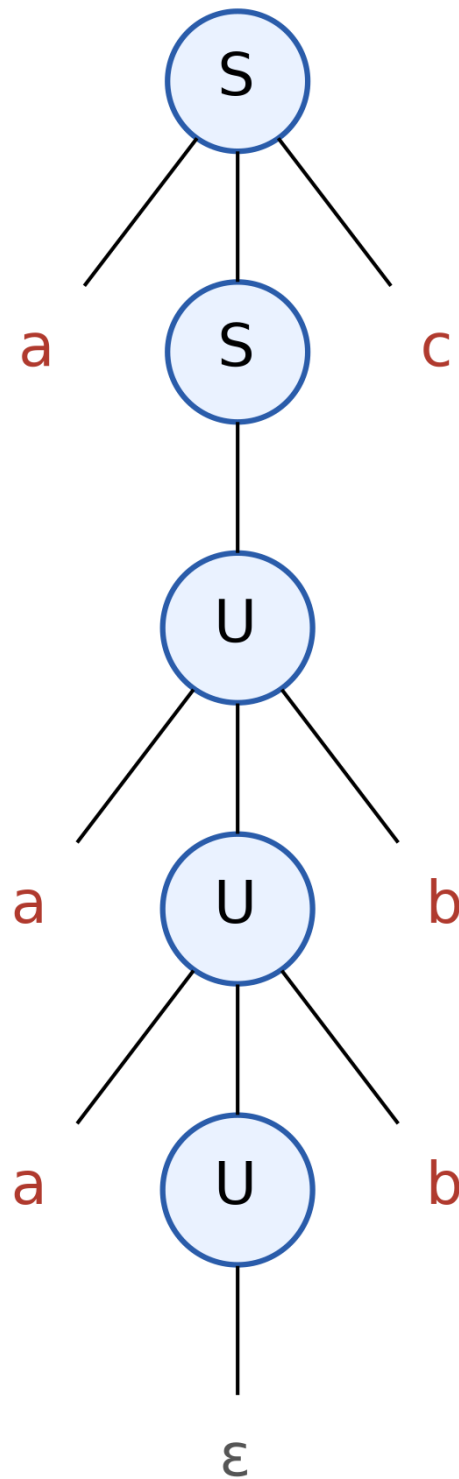


Figure 2.1 — Derivation (parse) tree for the word "aaabbc"

**Part C: Is the Grammar Ambiguous? Justification**

YES — the grammar IS ambiguous. A grammar is ambiguous if at least one string in its language has two or more distinct derivation/parse trees. Consider the short string "ac" ( $i = 1, j = 0, k = 1$ ). Because  $j = 0$ , this word

satisfies the boundary condition of BOTH branches simultaneously, and can be derived in three genuinely different ways:

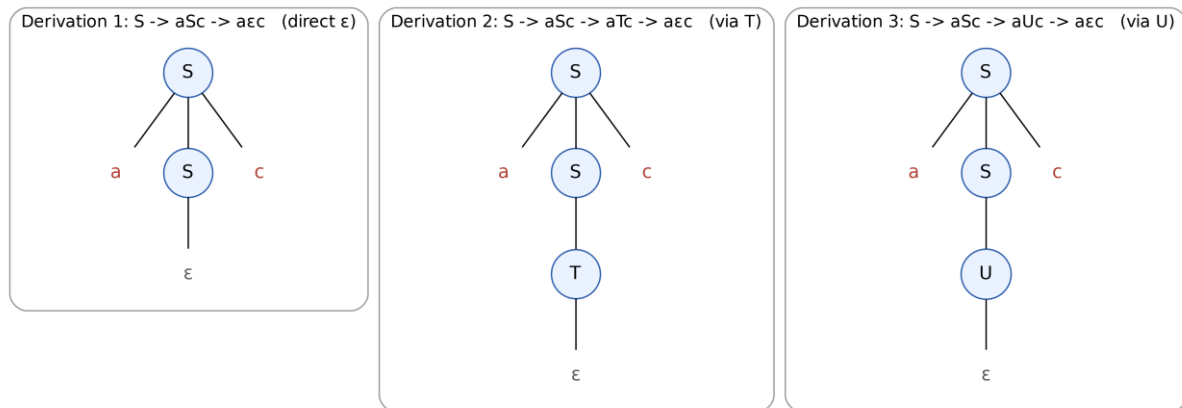


Figure 2.2 — Three distinct parse trees for the same word "ac", proving ambiguity

- Derivation 1 uses the production  $S \rightarrow \epsilon$  directly.
- Derivation 2 uses  $S \rightarrow T$  followed by  $T \rightarrow \epsilon$ .
- Derivation 3 uses  $S \rightarrow U$  followed by  $U \rightarrow \epsilon$ .

Since these are three structurally different parse trees (different production sequences / different intermediate non-terminals used) that all yield the identical final string "ac", the grammar satisfies the definition of ambiguity given in the question.

Justification: The root cause is that whenever  $j = 0$  (no b's in the word), the three "exit routes" out of S (direct  $\epsilon$ , via T with  $m = 0$ , via U with  $p = 0$ ) all collapse onto the same terminal string  $a^n c^n$ , yet the grammar keeps all three routes available instead of restricting to only one. A non-ambiguous grammar for this language would need to be redesigned so that only one of these routes is reachable for any given string, for example by removing the redundant direct  $S \rightarrow \epsilon$  production (since it is already reachable through T and U) — though note that even after removing it, the overlap between the T-branch and U-branch at  $j = 0$  (e.g., string "ac" would still be derivable via  $T \rightarrow \epsilon$  AND via  $U \rightarrow \epsilon$ ) means further restructuring would be required to fully remove ambiguity.



### Question 3 — $\epsilon$ -NFA for the Regular Expression $00^*1 + 11^*0$

Design a non-deterministic finite automaton with  $\epsilon$ -transitions to represent the language denoted by  $00^*1 + 11^*0$ .

#### Step 1: Decompose the Regular Expression

The expression is a union (+) of two concatenated sub-expressions:

- $R1 = 0 \cdot 0^* \cdot 1$  (one 0, followed by zero or more extra 0's, followed by a single 1)
- $R2 = 1 \cdot 1^* \cdot 0$  (one 1, followed by zero or more extra 1's, followed by a single 0)

$R = R1 + R2$ , so by Thompson's Union Rule we introduce a new start state with  $\epsilon$ -transitions branching into the machine for  $R1$  and the machine for  $R2$ .

#### Step 2: Apply Thompson's Construction Rules

- Basic Symbol Rule: build a single-edge machine for each literal symbol (0 or 1).
- Concatenation Rule (for  $00^*1$  and  $11^*0$ ): chain the machines in sequence; the  $0^* / 1^*$  part is realised as a self-loop, since "zero or more repeated symbols" is exactly a loop back on the same state.
- Union Rule (for the '+'): add a new start state  $q0$  with  $\epsilon$ -transitions to the start of each branch.

#### Step 3: Resulting $\epsilon$ -NFA

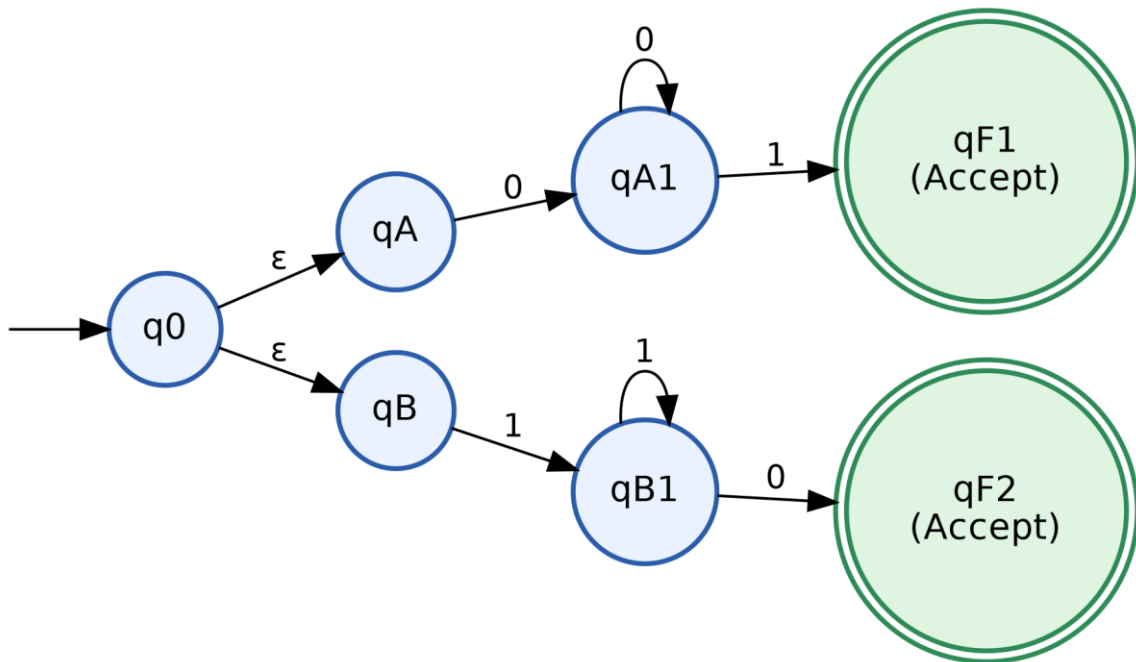


Figure 3.1 —  $\epsilon$ -NFA accepting  $L(00^*1 + 11^*0)$

#### Step 4: Formal Definition

$M = (Q, \Sigma, \delta, q0, F)$  where  $Q = \{q0, qA, qA1, qB, qB1, qF1, qF2\}$ ,  $\Sigma = \{0, 1\}$ , start state =  $q0$ ,  $F = \{qF1, qF2\}$ .

| State        | $\epsilon$ | 0     | 1     |
|--------------|------------|-------|-------|
| q0           | {qA, qB}   | —     | —     |
| qA           | —          | {qA1} | —     |
| qA1          | —          | {qA1} | {qF1} |
| qB           | —          | —     | {qB1} |
| qB1          | —          | {qF2} | {qB1} |
| qF1 (Accept) | —          | —     | —     |
| qF2 (Accept) | —          | —     | —     |

### Step 5: Trace Examples

| Input | Path Taken                                                                                                                         | Result                      |
|-------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| 01    | $q0 \rightarrow \epsilon \rightarrow qA \rightarrow 0 \rightarrow qA1 \rightarrow 1 \rightarrow qF1$                               | Accepted (matches $00^*1$ ) |
| 001   | $q0 \rightarrow \epsilon \rightarrow qA \rightarrow 0 \rightarrow qA1 \rightarrow 0 \rightarrow qA1 \rightarrow 1 \rightarrow qF1$ | Accepted (matches $00^*1$ ) |
| 10    | $q0 \rightarrow \epsilon \rightarrow qB \rightarrow 1 \rightarrow qB1 \rightarrow 0 \rightarrow qF2$                               | Accepted (matches $11^*0$ ) |
| 11    | $q0 \rightarrow \epsilon \rightarrow qB \rightarrow 1 \rightarrow qB1 \rightarrow 1 \rightarrow qB1$ (stuck, no accept on 1 alone) | Rejected                    |

Justification: Building the machine strictly by Thompson's rules (symbol  $\rightarrow$  star/concatenation  $\rightarrow$  union) guarantees, by construction, that the resulting  $\epsilon$ -NFA accepts exactly  $L(00^*1 + 11^*0)$ , and the branching structure (a single  $\epsilon$ -fork at q0) directly mirrors the '+' operator in the original expression, keeping the automaton small and easy to trace/verify.

## Question 4 — Closure Under Union & Regularity of $\{a^p \mid p \text{ prime}\}$

### Part (i): If $L_1$ and $L_2$ are Regular, then $(L_1 + L_2)$ is Regular

#### Step 1: What We Are Given

Since  $L_1$  is regular, there exists an NFA  $N_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$  with  $L(N_1) = L_1$ . Since  $L_2$  is regular, there exists an NFA  $N_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$  with  $L(N_2) = L_2$ . Assume  $Q_1$  and  $Q_2$  are disjoint (rename states if necessary).

#### Step 2: Construct a New Combined Automaton

Define a new NFA  $N = (Q, \Sigma, \delta, s_0, F)$  as follows:

- $Q = Q_1 \cup Q_2 \cup \{s_0\}$ , where  $s_0$  is a brand-new start state not in  $Q_1$  or  $Q_2$
- $\delta(s_0, \epsilon) = \{s_1, s_2\}$  ( $\epsilon$ -transitions to both original start states)
- $\delta(q, a) = \delta_1(q, a)$  for  $q \in Q_1$ , and  $\delta(q, a) = \delta_2(q, a)$  for  $q \in Q_2$  (all original transitions kept unchanged)
- $F = F_1 \cup F_2$  (accept in  $N$  if you would accept in EITHER original machine)

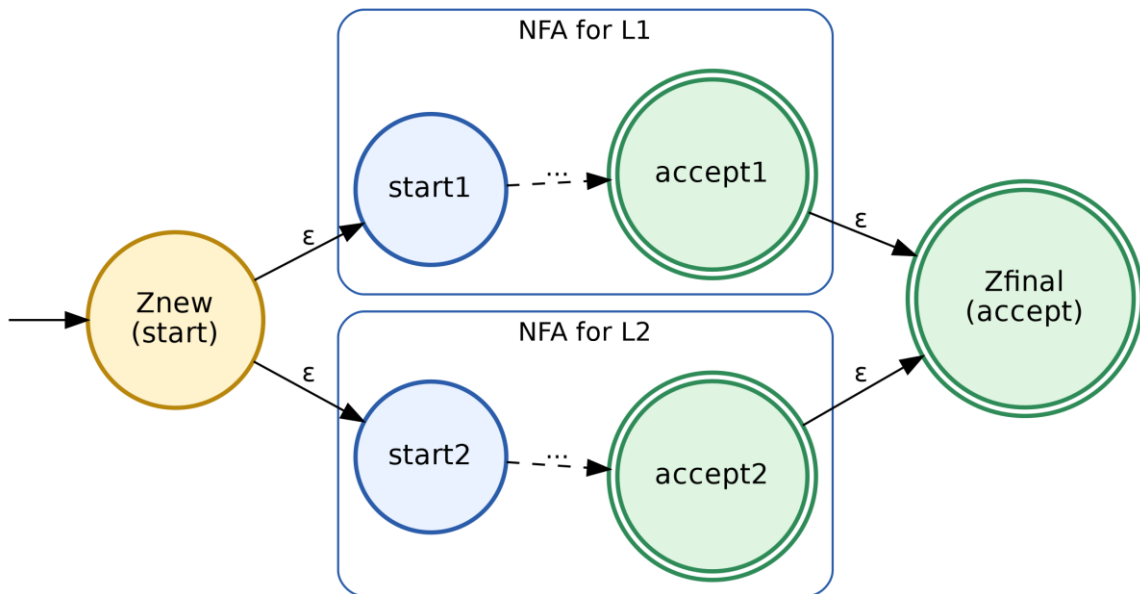


Figure 4.1 — Union construction: new start state  $\epsilon$ -branches into  $N_1$  and  $N_2$

#### Step 3: Prove $L(N) = L_1 \cup L_2$

Any string  $w$  is accepted by  $N$  if and only if, starting from  $s_0$ , some  $\epsilon$ -move leads into  $N_1$ 's start (and then  $w$  is processed entirely inside  $N_1$ , ending in  $F_1$ ) OR into  $N_2$ 's start (and then  $w$  ends in  $F_2$ ). Hence  $w \in L(N) \Leftrightarrow w \in L_1$  or  $w \in L_2 \Leftrightarrow w \in (L_1 \cup L_2)$ . So  $L(N) = L_1 + L_2$  exactly.

#### Step 4: Conclusion

N has a finite number of states ( $|Q_1| + |Q_2| + 1$ , which is finite because  $Q_1$  and  $Q_2$  are each finite) and finite, well-defined transitions — so N is a valid finite automaton. Since  $(L_1 + L_2)$  is recognized by a finite automaton N, by definition  $(L_1 + L_2)$  is a regular language.

Justification: This is precisely the standard closure-under-union proof: regularity is defined as "recognizable by some finite automaton," so exhibiting an explicit finite-automaton construction that recognizes exactly  $L_1 \cup L_2$  is a complete and rigorous proof of closure.

**Part (ii): Is  $L = \{ a^p \mid p \text{ is prime} \}$  Regular? — Justify**

NO — this language is NOT regular. We prove this using the Pumping Lemma for Regular Languages, by contradiction.

**Step 1: Assume L Is Regular**

If L were regular, the Pumping Lemma guarantees a pumping length  $n \geq 1$  such that every string  $w \in L$  with  $|w| \geq n$  can be split as  $w = xyz$ , with  $|xy| \leq n$ ,  $|y| \geq 1$ , and  $xy^iz \in L$  for every  $i \geq 0$ .

**Step 2: Choose a Suitable String**

Since there are infinitely many prime numbers (Euclid's theorem), we can always choose a prime number  $p$  such that  $p \geq n + 2$ . Let  $w = a^p$ , so  $w \in L$  and  $|w| = p \geq n$ .

**Step 3: Split and Pump**

Write  $w = xyz$  with  $|xy| \leq n$  and  $|y| = k \geq 1$  ( $k$  is some positive integer  $\leq n$ ). Since every symbol is 'a', we simply have  $|x| + |y| + |z| = p$ , and pumping  $y$  gives, for any  $i \geq 0$ :

$$x y^i z = a^{(p + (i - 1)k)}$$

**Step 4: Pick i to Force a Composite Length**

Choose  $i = p + 1$  (a valid non-negative integer). Then the resulting length is:

$$p + (i - 1)k = p + p \cdot k = p(1 + k)$$

Since  $p$  is prime (so  $p \geq 2$ ) and  $k \geq 1$  (so  $1 + k \geq 2$ ), the number  $p(1+k)$  is a product of two integers, each at least 2. Therefore  $p(1+k)$  is COMPOSITE, not prime.

**Step 5: Contradiction**

This means  $xy^{(p+1)}z = a^{(p(1+k))}$  has a composite (non-prime) number of a's, so  $xy^{(p+1)}z \notin L$ . But the Pumping Lemma requires  $xy^iz \in L$  for EVERY  $i \geq 0$ , including  $i = p+1$ . This is a direct contradiction.

**Step 6: Conclusion**

Since assuming L is regular leads to a contradiction, the assumption is false. Therefore:

**$L = \{ a^p \mid p \text{ is prime} \}$  is NOT a regular language.**

### Step 7: Numerical Illustration

| Item                      | Example Value                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------|
| Pumping length            | $n = 3$                                                                                       |
| Chosen prime $p \geq n+2$ | $p = 5$                                                                                       |
| String $w = a^p$          | aaaaa                                                                                         |
| Split (example)           | $x = \varepsilon, y = "a", z = "aaaa" \text{ (} k = 1 \text{)}$                               |
| Choose $i = p+1 = 6$      | $xy^iz = a^{(5+5 \times 1)} = a^{10}$                                                         |
| Is 10 prime?              | No — $10 = 2 \times 5$ , composite $\rightarrow a^{10} \notin L \rightarrow$<br>contradiction |

Justification: A finite automaton can only "count" using a bounded, fixed number of states, so it can never distinguish an unbounded, ever-changing set of prime lengths from the composite lengths between them. The pumping argument formalises this intuition: pumping a solution length by a multiple of that same length always produces a composite number, which is exactly why no DFA/NFA can recognize the primes-in-unary language.

## Question 5 — DFA Minimization

Consider the given DFA. Find equivalent states and construct the minimum-state automaton, draw it, write its transition table, and mark initial and final states.

### Step 1: The Given DFA

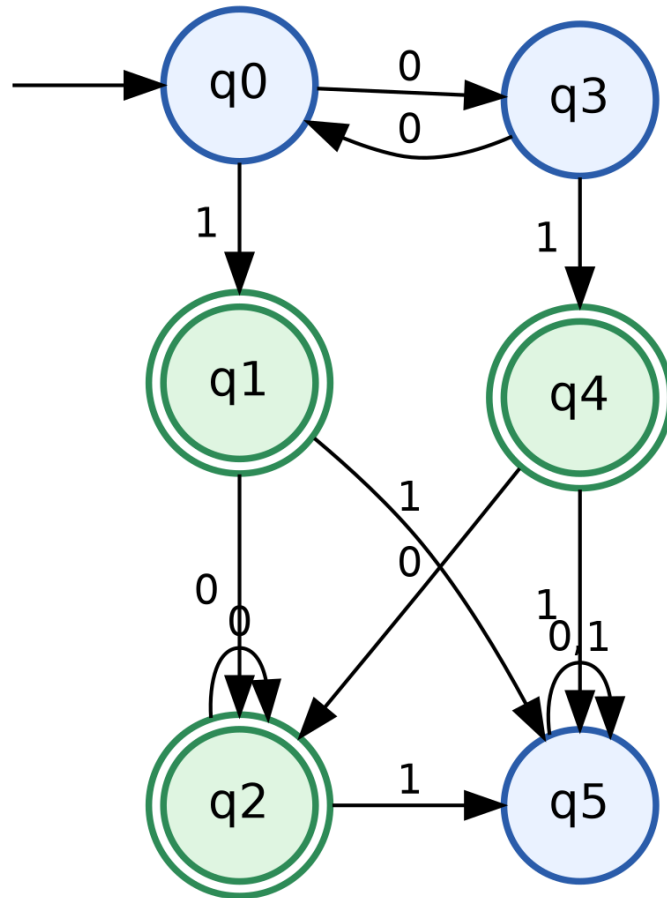


Figure 5.1 — Given DFA (6 states). Start =  $q_0$ . Accepting states =  $q_1, q_2, q_4$  (double circles).

### Step 2: Transition Table of the Given DFA

| State         | On 0  | On 1  | Type      |
|---------------|-------|-------|-----------|
| $q_0$ (Start) | $q_3$ | $q_1$ | Non-Final |
| $q_1$         | $q_2$ | $q_5$ | Final     |
| $q_2$         | $q_2$ | $q_5$ | Final     |
| $q_3$         | $q_0$ | $q_4$ | Non-Final |
| $q_4$         | $q_2$ | $q_5$ | Final     |

| State | On 0 | On 1 | Type      |
|-------|------|------|-----------|
| q5    | q5   | q5   | Non-Final |

### Step 3: Minimization — Partition Refinement (Table-Filling Method)

Partition P0 — split by final vs non-final status:

- $P0 = \{ \{q1, q2, q4\} \text{ (Final)} , \{q0, q3, q5\} \text{ (Non-Final)} \}$

Refine the Final group  $\{q1, q2, q4\}$  by checking where each state goes on each input, using P0 group labels (F = Final group, N = Non-Final group):

| State | On 0 → Group | On 1 → Group |
|-------|--------------|--------------|
| q1    | q2 → F       | q5 → N       |
| q2    | q2 → F       | q5 → N       |
| q4    | q2 → F       | q5 → N       |

All three states show the identical pattern (F, N) — no split needed.  $\{q1, q2, q4\}$  remains together.

Refine the Non-Final group  $\{q0, q3, q5\}$  the same way:

| State | On 0 → Group | On 1 → Group |
|-------|--------------|--------------|
| q0    | q3 → N       | q1 → F       |
| q3    | q0 → N       | q4 → F       |
| q5    | q5 → N       | q5 → N       |

q0 and q3 both show pattern (N, F) — they stay together. q5 shows pattern (N, N) — DIFFERENT from q0/q3, so q5 must SPLIT into its own group.

- $P1 = \{ \{q1, q2, q4\} , \{q0, q3\} , \{q5\} \}$

Check P1 again for further splits (one more refinement pass) using the new 3-group labels  $A = \{q1, q2, q4\}$ ,  $B = \{q0, q3\}$ ,  $C = \{q5\}$ :

- $q1: 0 \rightarrow q2 \in A, 1 \rightarrow q5 \in C \mid q2: 0 \rightarrow q2 \in A, 1 \rightarrow q5 \in C \mid q4: 0 \rightarrow q2 \in A, 1 \rightarrow q5 \in C \rightarrow$  all identical (A,C), no split
- $q0: 0 \rightarrow q3 \in B, 1 \rightarrow q1 \in A \mid q3: 0 \rightarrow q0 \in B, 1 \rightarrow q4 \in A \rightarrow$  both identical (B,A), no split

No group splits further. The partition is stable — this is the final answer:

- EQUIVALENT STATE CLASSES:  $A = \{q_0, q_3\}$ ,  $B = \{q_1, q_2, q_4\}$ ,  $C = \{q_5\}$

#### Step 4: Minimized DFA — Diagram

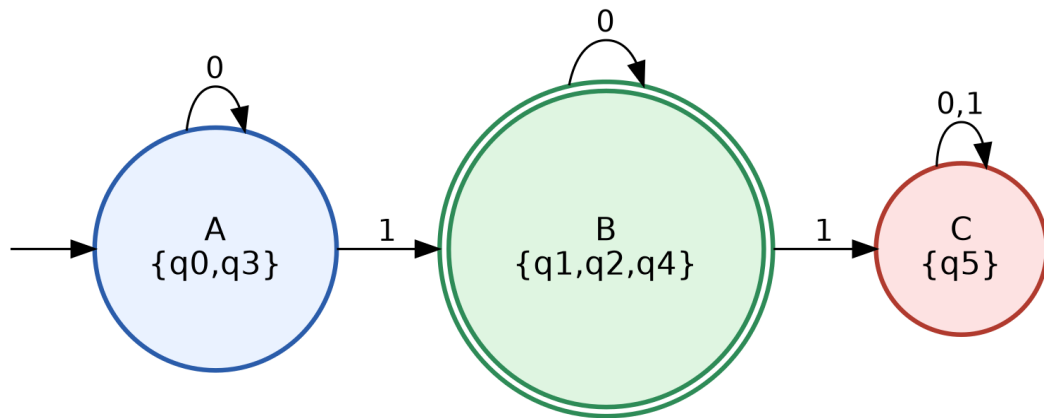


Figure 5.2 — Minimized DFA: 3 states. Initial state = A. Final state = B.

#### Step 5: Minimized Transition Table

| State | Equivalent To | On 0 | On 1 | Initial? | Final? |
|-------|---------------|------|------|----------|--------|
| A     | {q0, q3}      | A    | B    | Yes      | No     |
| B     | {q1, q2, q4}  | B    | C    | No       | Yes    |
| C     | {q5}          | C    | C    | No       | No     |

#### Step 6: Justification

- Correctness: Two states were merged only after formally proving they are indistinguishable — i.e., for every input string, they always produce the same accept/reject outcome (same target group on every symbol, recursively). This guarantees the minimized DFA accepts exactly the same language as the original 6-state DFA.
- Why  $q_5$  could not merge with  $\{q_0, q_3\}$ : even though  $q_5$  is non-final like  $q_0$  and  $q_3$ , on input '1'  $q_0/q_3$  move into the accepting group while  $q_5$  stays in its own non-accepting group forever — so a suitably chosen suffix (e.g., a single '1') would distinguish them, proving they are NOT equivalent.
- Efficiency gained: The state count was reduced from 6 to 3 (a 50% reduction), directly reducing the memory (flip-flops), transition logic, and power needed to implement this automaton in software or hardware, while preserving identical accept/reject behaviour for every possible input string.

Justification: The partition-refinement (table-filling) algorithm is a sound and complete method for DFA minimization — it merges only truly equivalent states and terminates with the unique (up to renaming) minimal DFA for the given regular language, as guaranteed by the Myhill–Nerode theorem.



